ENPM691 Homework 01: Measuring C Variable Sizes and Alignments

Nelay Sharma
University of Maryland
UID: 122295414
9/16/2025

***Abstract*—This homework demonstrates how C places data in memory. A simple program and the GNU debugger (GDB) were used to inspect addresses of primitive types, a mixed `struct`, and an array. Comparing consecutive addresses confirmed the sizes of each type. Inspecting a `struct` revealed how alignment and padding were observed which tells us how alignment and padding affect layout, while examining an array showed that the elements are stored contiguously. The results are presented in tables with concise explanations, making the process easy to follow along.**

I. Introduction

This homework explores three practical questions about C memory layout:

- **How big is each common primitive type?** (e.g., `int`, `long`, `float`, `double`, `char`)
- **How are fields placed inside a `struct`?** (i.e., including the padding the compiler inserts to keep things aligned)
- **Are arrays contiguous in memory?** (i.e., does each element sit right after the previous one?)

These topics matter for systems work and security because knowing where data is stored helps prevent crashes and errors. For example, if a programmer understands where and how the data is placed in memory, it becomes easier to avoid bugs, memory vulnerabilities, corrupted values, and security vulnerabilities. The approach here is simple: print the addresses, inspect them in GDB, and explain the results [1].

II. Methodology

- **Environment:** Kali Linux in VMware; `gcc` 14.3.0 and `gdb` 16.3 installed.
- **Code:** Arrays of `int`, `long`, `float`, `double`, and `char` were declared. Printing the address of one element next to the address of the next element shows the distance in bytes, which is the element size.
- **Struct:** A small mixed-type `struct` was examined with `gdb` using `ptype` and direct address prints to see where each field actually lands.
- **Compiler flags:** `-std=c11 -O0 -g`
- **Debugger steps:** `break main`, `run`, `info locals`, `p &var`, and `ptype` for type/offset info.
- **Data collected:** Printed addresses of consecutive elements for each type, the byte differences between them, and GDB field offsets for a mixed struct.

III. Results

A. Primitive Variables

To confirm sizes, the program prints two back-to-back addresses for each type and notes the difference. That difference is the size of that type on this machine. The exact addresses change each run (i.e., due to ASLR, which randomizes memory layout due to a security feature in modern operating systems), but the *difference* stays stable.

TABLE I
Sizes and Alignments of Primitive Types

Type	Address	Bytes
int	0x7fffee0cd3f8	4
long	0x7fffee0cd3e0	8
float	0x7fffee0cd3d8	4
double	0x7fffee0cd3c0	8
char	0x7fffee0cd3be	1

On a 64-bit LP64 system, the results line up with the expectations: `int` and `float` are 4 bytes; `long` and `double` are 8 bytes; `char` is 1 byte.

B. Struct Layout

A mixed `struct` was inspected to see where each field ends up. The first field starts at the beginning of the `struct` (offset 0). Later fields may not follow immediately; the compiler inserts small gaps (ie., padding) so each field starts on an address that matches its alignment requirements. This is why an `int` may begin at 4, `long` at 8, `float` at 16, and so on. The table below lists the observed offsets from one run.

TABLE II
Struct Field Offsets

Field	Offset (bytes)
c (char[2])	0
i (int[2])	4
l (long[2])	8
f (float[2])	16
d (double[2])	24
arr (int[4])	32

The idea here is that the compiler is not being “wasteful”, rather it is actually keeping larger types properly aligned (via padding and alignment rules) so the CPU can access them efficiently and correctly.

C. Array Layout

In C, arrays are stored contiguously, meaning each element is placed directly after the previous one in memory. In the test program, the printed addresses for `int arr[4]` show that each step from one element to the next is exactly 4 bytes (which is increased each time), and matches the size of an `int` from one element to the next. This proves the array is laid out on a contiguous block of memory, which makes indexing and pointer arithmetic reliable.

TABLE III
ARRAY CONTIGUITY FOR `INT ARR[4]`

Element	Address	Δ
arr[0]	0x7fffee0cd3a0	–
arr[1]	0x7fffee0cd3a4	+4
arr[2]	0x7fffee0cd3a8	+4
arr[3]	0x7fffee0cd3ac	+4

IV. DISCUSSION AND CONCLUSION

The small program and GDB session make the memory story visible:

- **Primitive sizes** matched the platform's 64-bit ABI expectations.
- **Structs** showed padding between fields so that each one starts at a "good" address for that type; This is normal and driven by hardware alignment rules.
- **Arrays** confirmed contiguity, each element lives right after the previous one with a fixed step equal to the element size.

These observations highlight the importance of understanding memory layout in C. Primitive sizes came out exactly as expected on a 64-bit system, and reinforces that the compiler sticks to ABI rules. The struct test revealed that padding is not wasted space, but rather an intentional mechanism to preserve proper alignment and ensure efficient CPU access. In addition, arrays reinforced the guarantee that elements are stored contiguously in memory, with each one sitting directly after the previous one at a fixed step size.

For day-to-day coding, while this may appear like a small detail, this exercise serves as a reminder that even low-level details (i.e., alignment, padding, data placement) influence program correctness and performance, as improper assumptions about memory layout may lead to bugs, crashes, or degraded performance.

ACKNOWLEDGEMENTS

Special thanks to Dr. Lewis Collier and TA Ilya Yatsenko for assistance in answering questions and guidance for this homework assignment.

REFERENCES

[1] ENPM691 Lecture 01, Homework Assignment Overview.

APPENDIX A
MAIN SOURCE CODE

```
#include <stdio.h>
#include <stddef.h>
// hw1_primitives_arrays.c

int main(void){
    // Declare arrays of each primitive type
    int i[2]; long l[2]; float f[2]; double d[2]; char c[2];
    int arr[4];

    // Print the address of the first int element (i[0])
    // Then the difference in bytes between i[1] and i[0] via subtracting
    // Which equals to the size of an int on this system

    printf("int:    &i0=%p =%td\n", (void*)&i[0], (ptrdiff_t)((char*)&i[1]-(char*)&i[0]));
    printf("long:   &l0=%p =%td\n", (void*)&l[0], (ptrdiff_t)((char*)&l[1]-(char*)&l[0]));
    printf("float:  &f0=%p =%td\n", (void*)&f[0], (ptrdiff_t)((char*)&f[1]-(char*)&f[0]));
    printf("double: &d0=%p =%td\n", (void*)&d[0], (ptrdiff_t)((char*)&d[1]-(char*)&d[0]));
    printf("char:   &c0=%p =%td\n", (void*)&c[0], (ptrdiff_t)((char*)&c[1]-(char*)&c[0]));


    // Print the address of four consecutive int elements, shows arrays stored contiguously
    printf("arr: &a0=%p &a1=%p &a2=%p &a3=%p\n",
           (void*)&arr[0], (void*)&arr[1], (void*)&arr[2], (void*)&arr[3]);
    return 0;
}
```

APPENDIX B
BUILD/RUN SCRIPTS

```
$ gcc -std=c11 -O0 -g hw1_primitives_arrays.c -o hw1_pa
$ ./hw1_pa
```

APPENDIX C
DEBUGGER OUTPUT

```
Breakpoint 1, main () at hw1_primitives_arrays.c:9
i = {-8704, 32767}
l = {0, 140737354024832}
f = {0, 0}
d = {0, 0}
c = "\000"
arr = {26, 36, 0, 0}
$1 = (int (*)[2]) 0x7fffffffdd68
$2 = (long (*)[2]) 0x7fffffffdd50
$3 = (float (*)[2]) 0x7fffffffdd48
$4 = (double (*)[2]) 0x7fffffffdd30
$5 = (char (*)[2]) 0x7fffffffdd2e
$6 = (int (*)[4]) 0x7fffffffdd10
type = int [2]
type = long [2]
type = float [2]
type = double [2]
type = char [2]
type = int [4]
```

APPENDIX D
RAW PROGRAM OUTPUT (ONE RUN)

```
int:    &i0=0x7fffee0cd3f8 =4
long:   &l0=0x7fffee0cd3e0 =8
float:  &f0=0x7fffee0cd3d8 =4
double: &d0=0x7fffee0cd3c0 =8
char:   &c0=0x7fffee0cd3be =1
arr: &a0=0x7fffee0cd3a0 &a1=0x7fffee0cd3a4 &a2=0x7fffee0cd3a8 &a3=0x7fffee0cd3ac
```